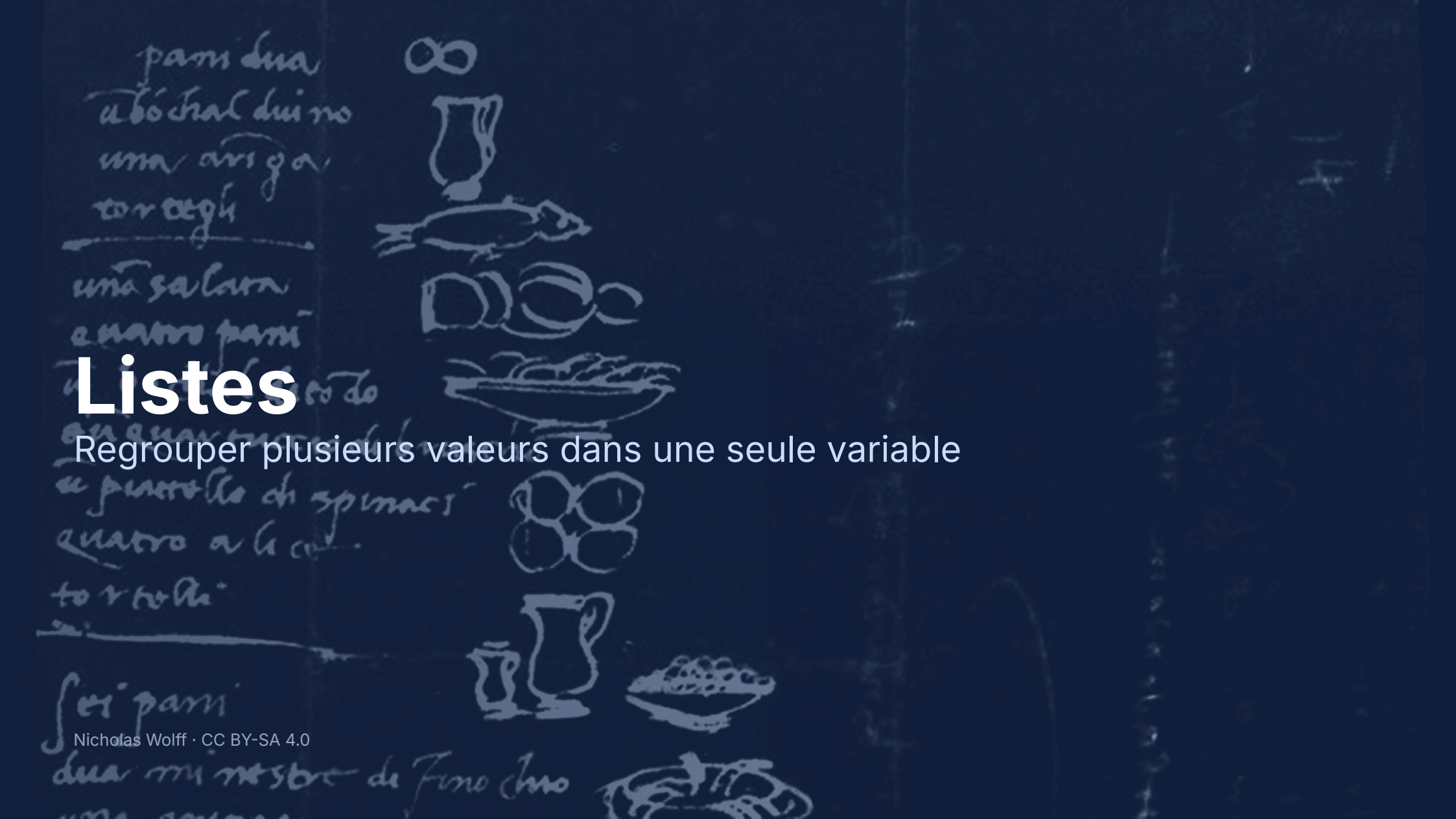


Listes

Regrouper plusieurs valeurs dans une seule variable

Nicholas Wolff · CC BY-SA 4.0

Sommaire

1

Utilité, définition

À quoi servent les listes
Ce qu'est une liste

2

Rappels sur les types de données

Les quatre types vus jusqu'à maintenant

3

Quelques opérations sur les listes

- Créer une liste
- Y ajouter un élément
- En enlever un élément
- Connaître sa longueur
- Savoir si elle est vide
- Savoir si un élément est dans une liste

4

Itération avec les listes

On connaît déjà l'idée d'itération, qui consiste à répéter la même opération jusqu'à ce qu'on ait fini. Les listes se prêtent particulièrement bien à cette utilisation.

Utilité

On se sert de listes constamment dans la vie courante.
Exemples : liste de courses, playlist musicale, liste d’invités, liste d’attente.

Une liste permet de stocker plusieurs informations dans un seul ensemble et d’y faire référence en utilisant une variable.
Exemple : une liste de courses regroupe tous les articles à acheter, au lieu de les retenir un par un.

Les listes vont particulièrement bien s’intégrer avec l’idée d’itération que nous connaissons.
Exemple : parcourir une liste d’élèves pour afficher leurs notes.

Définition

Une liste est une *structure de données* :

- **Ordonnée.**
  `[1, 2]` et `[2, 1]` sont deux listes différentes.
- **Qui peut contenir plusieurs fois le même élément.**
  `[1, 1, 2]` est une liste correcte.
- **Sans trous.**
  Une liste contient tous les éléments entre le premier et le dernier.

On parle d'une liste d'*éléments* : par exemple, 1 est un *élément* des trois listes ci-dessus.

Une structure de données est une manière de regrouper des éléments entre eux, pour pouvoir parler de choses plus complexes que simplement des nombres, des chaînes de caractères et des booléens.

Sommaire

1

Utilité, définition

À quoi servent les listes

Ce qu'est une liste

2

Rappels sur les types de données

Les quatre types vus jusqu'à maintenant

3

Quelques opérations sur les listes

- Créer une liste
- Y ajouter un élément
- En enlever un élément
- Connaître sa longueur
- Savoir si elle est vide
- Savoir si un élément est dans une liste

4

Itération avec les listes

On connaît déjà l'idée d'itération, qui consiste à répéter la même opération jusqu'à ce qu'on ait fini. Les listes se prêtent particulièrement bien à cette utilisation.

Quatre types de données vus jusqu'à maintenant

Les **nombres entiers**. En Python : `int`.

```
0   1   2346   -235098
```

Les **nombres décimaux**. En Python : `float`. Le séparateur décimal est un point.

```
0.5   0.0   .25   3.1415
```

Les **chaînes de caractères**. En Python : `str`. Elles représentent des suites de lettres et de symboles. Elles sont délimitées par des guillemets `"`.

```
"bob"   "rouge"   "1"   ""
```

Les **booléens**. En Python : `bool`. Il y a seulement deux valeurs booléennes possibles :

```
True   False
```

On les manipule parfois directement, mais elles sont surtout utilisées dans les conditions des `if` et des `while`. Par exemple :

```
>>> x = 4
>>> x > 2
True
```

```
>>> x = 1
>>> x > 2
False
```

Sommaire

1

Utilité, définition

À quoi servent les listes
Ce qu'est une liste

2

Rappels sur les types de données

Les quatre types vus jusqu'à maintenant

3

Quelques opérations sur les listes

- Créer une liste
- Y ajouter un élément
- En enlever un élément
- Connaître sa longueur
- Savoir si elle est vide
- Savoir si un élément est dans une liste

4

Itération avec les listes

On connaît déjà l'idée d'itération, qui consiste à répéter la même opération jusqu'à ce qu'on ait fini. Les listes se prêtent particulièrement bien à cette utilisation.

Créer une liste

On note une liste avec des crochets, en séparant les éléments par des virgules.
Sur un clavier Mac : ⌥ Alt + 5 pour [et ⌥ Alt + 6 pour].

Une liste de 7 nombres :

```
nombres = [3, 6, 2, 8, 1, 9, 5]
```

Une liste vide :

```
lst = []
```

Une liste contenant un élément :

```
a = [42.5]
```

Une liste de chaînes de caractères :

```
équipes = ["Liverpool", "Marseille", "Ajax"]
```

Une *liste de listes* de nombres :

```
tableau = [[4, 6], [3, -1]]
```

On peut écrire une longue liste sur plusieurs lignes :

```
zz = [
    1, 2, 3, 4, 5, 6, 7, 8, 9, 10,
    11, 12, 13, 14, 15, 16, 17
]
```

Ajouter un élément à une liste

On peut ajouter un élément à la **fin** d'une liste avec append :

```
>>> lst = [3, 6, 2, 8, 1, 9, 5]
>>> lst.append(19)
>>> print(lst)
[3, 6, 2, 8, 1, 9, 5, 19]
```

Pour s'en souvenir, on peut remarquer que *append* se termine avec « end », ce qui veut dire « fin ».

On peut ajouter un élément au **début** de la liste avec `insert(0, …)` :

```
>>> lst = [3, 6, 2, 8, 1, 9, 5]
>>> lst.insert(0, 19)
>>> print(lst)
[19, 3, 6, 2, 8, 1, 9, 5]
```

- On voit que les éléments qui suivent l'élément inséré ont été décalés vers la droite.
- `insert` permet en réalité de faire plus de choses. Ici, le `0` veut dire « au début ».

Attention à cette syntaxe, qu'on voit pour la première fois :
`variable_contenant_la_liste``.append(…)`

Enlever un élément d'une liste 1/2

On peut utiliser pop pour *enlever* un élément de la liste **et** récupérer sa valeur.

`pop()` pour enlever depuis la *fin* de la liste :

```
>>> lst = [3, 6, 2, 8, 1, 9, 7, 5]
>>> e = lst.pop()
>>> print(e)
5
>>> print(lst)
[3, 6, 2, 8, 1, 9, 7]
```

`pop(0)` pour enlever depuis le *début* de la liste :

```
>>> lst = [3, 6, 2, 8, 1, 9, 7, 5]
>>> e = lst.pop(0)
>>> print(e)
3
>>> print(lst)
[6, 2, 8, 1, 9, 7, 5]
```

Enlever un élément d'une liste 2/2

On peut utiliser pop pour *enlever* un élément de la liste **et** récupérer sa valeur.

Supprimer le dernier élément sans le récupérer (c'est rarement ce qu'on veut) :

```
>>> lst = [3, 6, 2, 8, 1, 9, 7, 5]
>>> lst.pop()
>>> print(lst)
[3, 6, 2, 8, 1, 9, 7]
```

On a perdu le 5 : c'est comme si on l'avait jeté à la poubelle.

pop d'une liste vide donne une erreur :

```
>>> lst = []
>>> lst.pop()
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
IndexError: pop from empty list
```

Longueur de liste, savoir si une liste est vide

On peut obtenir la longueur d'une liste avec la fonction `len( )` :

```
>>> lst = [3, 6, 2, 8, 1, 9, 5]
>>> longueur = len(lst)
>>> print(longueur)
7

>>> lst = []
>>> print(len(lst))
0
```

- Comme dans tous les programmes que nous faisons, nous utilisons des exemples, mais il n'est pas question de compter les éléments (ici d'abord 7, ensuite 0) et d'écrire ces valeurs dans le programme.
- Le mot `len` est une version raccourcie de « length » (longueur).

On peut utiliser une liste dans une *condition* pour savoir si elle est vide :

```
>>> lst = [3, 6, 2, 8, 1, 9, 5]
>>> if lst:
...     print("Pas vide")
... else:
...     print("Vide")
...
Pas vide
```

La condition `len(lst) > 0` est équivalente, mais juste écrire `lst` dans une condition est plus simple.

Savoir si un élément est dans une liste

`élément in liste`

s'évalue soit en `True`, soit en `False`. On s'en servira partout où une condition est attendue, par exemple dans `if condition:` et `while condition:`.

Exemple :

```
nombres_gagnants = [82, 93, 58, 33, 3]
n = int(input("votre nombre : "))
if n in nombres_gagnants:
    print("BINGO!")
```

Exercice 1 : manipulation de listes

Exercice · 5 min

Écrire un programme qui :

1) crée une liste `amis`, avec le prénom de 4 de vos amis proches ;
2) ajoute ensuite `"nico"` à la fin de la liste ;
3) enlève le premier élément de la liste ;
4) affiche la liste résultante ;
5) affiche la longueur de la liste résultante ;
6) affiche si `"nico"` est dans la liste des amis (il doit afficher `True` ou `False`).

Indications :

- ⌥ Alt + 5 pour `[`
- ⌥ Alt + 6 pour `]`
- `lst = […, …, …]`
- `lst.insert(0, …)` et `lst.append(…)`
- `lst.pop(0)` et `lst.pop()`
- `len(lst)`
- `… in lst`

Exercice 1 : manipulation de listes

Écrire un programme qui :

1) *crée une liste `amis`, avec le prénom de 4 de vos amis proches ;*
2) *ajoute ensuite `"nico"` à la fin de la liste ;*
3) *enlève le premier élément de la liste ;*
4) *affiche la liste résultante ;*
5) *affiche la longueur de la liste résultante ;*
6) *affiche si `"nico"` est dans la liste des amis (il doit afficher `True` ou `False`).*

```python
# Création de la liste avec 4 amis
amis = ["alex", "marie", "sara", "tom"]

# Ajout de "nico" à la fin
amis.append("nico")

# Suppression du premier élément
amis.pop(0)

# Affichage de la liste résultante
print("Liste finale :", amis)

# Affichage de la longueur de la liste
print("Longueur de la liste :", len(amis))

print("nico est un ami ? ", "nico" in amis)
```

Le programme affiche :

```
Liste finale : ['marie', 'sara', 'tom', 'nico']
Longueur de la liste : 4
nico est un ami ?  True
```

Le programme : [manipulation-de-listes-solution.py]

Sommaire

1

Utilité, définition

À quoi servent les listes
Ce qu'est une liste

2

Rappels sur les types de données

Les quatre types vus jusqu'à maintenant

3

Quelques opérations sur les listes

- Créer une liste
- Y ajouter un élément
- En enlever un élément
- Connaître sa longueur
- Savoir si elle est vide
- Savoir si un élément est dans une liste

4

Itération avec les listes

On connaît déjà l'idée d'itération, qui consiste à répéter la même opération jusqu'à ce qu'on ait fini. Les listes se prêtent particulièrement bien à cette utilisation.

Itération des éléments d'une liste

On combine `pop()` et le fait qu'on sait vérifier si une liste est vide :

```
lst = [5, 7, 35, -11]
while lst:
    e = lst.pop()
    print("Élément :", e, "| Liste :", lst)
```

Itération des éléments d'une liste

On combine `pop()` et le fait qu'on sait vérifier si une liste est vide :

```
1 lst = [5, 7, 35, -11]
2 while lst:
3     e = lst.pop()
4     print("Élément :", e, "| Liste :", lst)
```

```
Élément : -11 | Liste : [5, 7, 35]
```

Itération des éléments d'une liste

On combine `pop()` et le fait qu'on sait vérifier si une liste est vide :

```
1 lst = [5, 7, 35, -11]
2 while lst:
3     e = lst.pop()
4     print("Élément :", e, "| Liste :", lst)
```

```
Élément : -11 | Liste : [5, 7, 35]
Élément : 35 | Liste : [5, 7]
Élément : 7 | Liste : [5]
Élément : 5 | Liste : []
```

Exercice 2 : top des villes

Exercice · 8 min

Étant donné cette liste :

```
villes = [
    "Zürich", "Genève", "Bâle", "Lausanne",
    "Winterthur", "Lucerne", "Saint-Gall",
    "Lugano", "Bienne", "Berne",
]
```

1) Écrire un premier programme qui itère dans la liste et affiche le nom de chaque ville sur une ligne.

2) Faire une copie de ce premier programme, et l'appeler « villes numérotées ». Ce deuxième programme affiche :

```
1 Zürich
2 Genève
3 Bâle
4 Lausanne
5 Winterthur
6 Lucerne
7 Saint-Gall
8 Lugano
9 Bienne
10 Berne
```

Indications pour 1)

- Utiliser la technique pour itérer sur une liste.

Indications pour 2)

- Utiliser la technique pour itérer sur une liste.
- Se souvenir de la technique pour compter des choses, vue dans le cours « Itération : while ».
- Combiner les deux techniques. Attention : ce qui détermine quand on s'arrête, c'est quand la liste est vide, et pas le numéro associé à la ville.

Le fichier de base avec la liste des villes : top-villes.py

Exercice 2 : top des villes

1) Écrire un premier programme qui itère dans la liste et affiche le nom de chaque ville sur une ligne.

```
villes = [
    "Zürich", "Genève", "Bâle", "Lausanne",
    "Winterthur", "Lucerne", "Saint-Gall",
    "Lugano", "Bienne", "Berne",
]

while villes:
    print(villes.pop(0))
    # On pourrait aussi faire en deux lignes:
    # ville = villes.pop(0)
    # print(ville)
```

Le programme : [villes-sans-numeros-solution.py]

Exercice 2 : top des villes

2) Faire une copie de ce premier programme, et l'appeler « villes numérotées ». Ce deuxième programme affiche :

```
1 Zürich
2 Genève
3 Bâle
4 Lausanne
5 Winterthur
6 Lucerne
7 Saint-Gall
8 Lugano
9 Bienne
10 Berne
```

```python
villes = [
    "Zürich", "Genève", "Bâle", "Lausanne",
    "Winterthur", "Lucerne", "Saint-Gall",
    "Lugano", "Bienne", "Berne",
]

i = 1

while villes:
    print(i, villes.pop(0))
    i = i + 1
```

Le programme : villes-numerotees-solution.py

Exercice 3 : afficher des paires à partir de deux listes

Exercice · 6 min

Étant donné deux listes *de taille identique*, par exemple :

```
les_mois = [
    "janvier", "février", "mars", "avril",
    "mai", "juin", "juillet", "août",
    "septembre", "octobre", "novembre", "décembre",
]

nombres_de_jours = [31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31]
```

Écrire un programme qui parcourt les deux listes *en même temps* et affiche sur chaque ligne le nom du mois, puis deux points, puis le nombre de jours du mois.

Le résultat doit ressembler à ceci :

```
janvier : 31
février : 28
mars : 31
avril : 30
mai : 31
juin : 30
juillet : 31
août : 31
septembre : 30
octobre : 31
novembre : 30
décembre : 31
```

Le fichier de base avec les deux listes : paires.py

Exercice 3 : afficher des paires à partir de deux listes

Écrire un programme qui parcourt les deux listes en même temps *et affiche sur chaque ligne le nom du mois, puis deux points, puis le nombre de jours du mois.*

```
les_mois = [
    "janvier", "février", "mars", "avril",
    "mai", "juin", "juillet", "août",
    "septembre", "octobre", "novembre", "décembre",
]

nombres_de_jours = [31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31]

while les_mois:
    mois = les_mois.pop(0)
    jours = nombres_de_jours.pop(0)  # Il n'y a pas de risque de tomber sur une liste vide ici
                                     # parce que les deux listes ont la même longueur
    print(mois, ":", jours)
```

Le programme : paires-solution.py

Exercice 4 : séparer des paires

Exercice · 7 min

Étant donné une liste contenant des paires d'informations (un mois, puis le nombre de jours, puis un mois, etc.) :

```
mois_et_jours = [
    "janvier", 31, "février", 28, "mars", 31, "avril", 30,
    "mai", 31, "juin", 30, "juillet", 31, "août", 31,
    "septembre", 30, "octobre", 31, "novembre", 30, "décembre", 31,
]
```

Écrire un programme qui affiche le nombre de jours de chaque mois :

```
31 jours en janvier
28 jours en février
31 jours en mars
30 jours en avril
31 jours en mai
30 jours en juin
31 jours en juillet
31 jours en août
30 jours en septembre
31 jours en octobre
30 jours en novembre
31 jours en décembre
```

Le fichier de base avec la liste : separer-des-paires.py

Conseils

- Évidemment (et comme d'habitude), votre programme doit continuer de fonctionner même avec d'autres listes : il ne faut pas qu'il y ait de nombres fixés dans votre code.
- Réaliser le programme *graduellement*. Si ça vous aide, vous pouvez commencer par écrire une toute petite partie du programme, sans boucle, éventuellement en faisant des copier-coller. Une fois que vous êtes sûr·es de votre approche, introduire la boucle.

Exercice 4 : séparer des paires

Écrire un programme qui affiche le nombre de jours de chaque mois :

```python
mois_et_jours = [
    "janvier", 31, "février", 28, "mars", 31, "avril", 30,
    "mai", 31, "juin", 30, "juillet", 31, "août", 31,
    "septembre", 30, "octobre", 31, "novembre", 30, "décembre", 31,
]

while mois_et_jours:
    mois = mois_et_jours.pop(0)
    jours = mois_et_jours.pop(0)
    print(jours, "jours en", mois)
```

Le programme : [separer-des-paires-solution.py]

Le programme affiche :

```
31 jours en janvier
28 jours en février
31 jours en mars
30 jours en avril
31 jours en mai
30 jours en juin
31 jours en juillet
31 jours en août
30 jours en septembre
31 jours en octobre
30 jours en novembre
31 jours en décembre
```

Exercice 5 : afficher un élément sur deux

Exercice · 5 min

Étant donné une liste contenant des paires d'informations (un mois, puis le nombre de jours, puis un mois, etc.) :

```
mois_et_jours = [
    "janvier", 31, "février", 28, "mars", 31, "avril", 30,
    "mai", 31, "juin", 30, "juillet", 31, "août", 31,
    "septembre", 30, "octobre", 31, "novembre", 30, "décembre", 31,
]
```

Écrire un programme qui affiche seulement le nom des mois :

```
janvier
février
mars
avril
mai
juin
juillet
août
septembre
octobre
novembre
décembre
```

Le fichier de base avec la liste : afficher-un-sur-deux.py

Conseils

- Évidemment (et comme d'habitude), votre programme doit continuer de fonctionner même avec d'autres listes : il ne faut pas qu'il y ait de nombres fixés dans votre code.
- Réaliser le programme *graduellement*. Si ça vous aide, vous pouvez commencer par écrire une toute petite partie du programme, sans boucle, éventuellement en faisant des copier-coller. Une fois que vous êtes sûr·es de votre approche, introduire la boucle.

Exercice 5 : afficher un élément sur deux

Écrire un programme qui affiche seulement le nom des mois :

```python
mois_et_jours = [
    "janvier", 31, "février", 28, "mars", 31, "avril", 30,
    "mai", 31, "juin", 30, "juillet", 31, "août", 31,
    "septembre", 30, "octobre", 31, "novembre", 30, "décembre", 31,
]

while mois_et_jours:
    print(mois_et_jours.pop(0))

    # On jette le nombre de jours du mois,
    # comme ça le prochain élément qu'on trouvera
    # dans la liste sera un mois
    mois_et_jours.pop(0)
```

Le programme affiche :

```
janvier
février
mars
avril
mai
juin
juillet
août
septembre
octobre
novembre
décembre
```

Le programme : [afficher-un-sur-deux-solution.py]

Exercice 6 : longs mois

Exercice · 5 min

Étant donné une liste contenant des paires d'informations (un mois, puis le nombre de jours, puis un mois, etc.) :

```
mois_et_jours = [
    "janvier", 31, "février", 28, "mars", 31, "avril", 30,
    "mai", 31, "juin", 30, "juillet", 31, "août", 31,
    "septembre", 30, "octobre", 31, "novembre", 30, "décembre", 31,
]
```

Écrire un programme qui affiche le nom des mois qui contiennent plus de 30 jours :

```
janvier
mars
mai
juillet
août
octobre
décembre
```

Le fichier de base avec la liste : longs-mois.py

Conseils

- Évidemment (et comme d'habitude), votre programme doit continuer de fonctionner même avec d'autres listes : il ne faut pas qu'il y ait de nombres fixés dans votre code.
- Dans ce programme, il y a manifestement une itération sur une liste : on va donc utiliser une boucle.
- Ce programme est similaire au programme précédent.
- La différence, c'est qu'ici il y a aussi un traitement conditionnel (on n'affiche un mois que si son nombre de jours est suffisant). Il va falloir *combiner* `if` et `while` pour arriver au résultat.

Exercice 6 : longs mois

Écrire un programme qui affiche le nom des mois qui contiennent plus de 30 jours :

```
mois_et_jours = [
    "janvier", 31, "février", 28, "mars", 31, "avril", 30,
    "mai", 31, "juin", 30, "juillet", 31, "août", 31,
    "septembre", 30, "octobre", 31, "novembre", 30, "décembre", 31,
]

while mois_et_jours:
    mois = mois_et_jours.pop(0)
    jours = mois_et_jours.pop(0)
    if jours > 30:
        print(mois)
```

Le programme affiche :

```
janvier
mars
mai
juillet
août
octobre
décembre
```

Le programme : [longs-mois-solution.py]

Sources

Images

Michel-Ange – une liste de courses, Casa Buonarroti, Florence (vers 1518, domaine public)
commons.wikimedia.org